

**OBJECT-ORIENTED
PROGRAMMING**
CL - 1004

LAB 02

Exploring Pointers, Dynamic Memory
Allocation, and Structs in C++.

National University of Computer and Emerging Sciences–NUCES – Karachi

Instructor: **Talha Shahid**

Contents

1. Pointers in C++	3
1.1. Single Pointer	3
1.2. 2D-Pointer.....	4
1.3. Dynamic Memory Allocation	5
1.3.1. new Keyword	5
1.3.2. delete Keyword	6
1.3.3. Example 01: Code for 2D-Pointer using Dynamic Memory Allocation	7
1.3.4. Example 02: Code for 2D-Pointer using Dynamic Memory Allocation with Pointer Arithmetic ..	8
2. structs in C++	10
2.1. Declaring a struct in C++.....	10
2.2. Defining a struct Variable in C++	10
2.3. Accessing Members of struct in C++	11
2.4. Real World Use Case of struct in C++	11

1. Pointers in C++

A Pointer is a variable whose content is a memory address. In C++, a pointer is a variable that stores the memory address of another variable. Pointers are one of the fundamental concepts in C++ that allow for dynamic memory allocation, passing variables by reference.

1.1. Single Pointer

A single pointer refers to a pointer variable that holds the address of a single variable of a specific type. It points to a single memory location. To declare a single pointer variable, you need to specify the data type, an asterisk symbol (*) and the name of the pointer variable.

```
dataType *ptrName
```

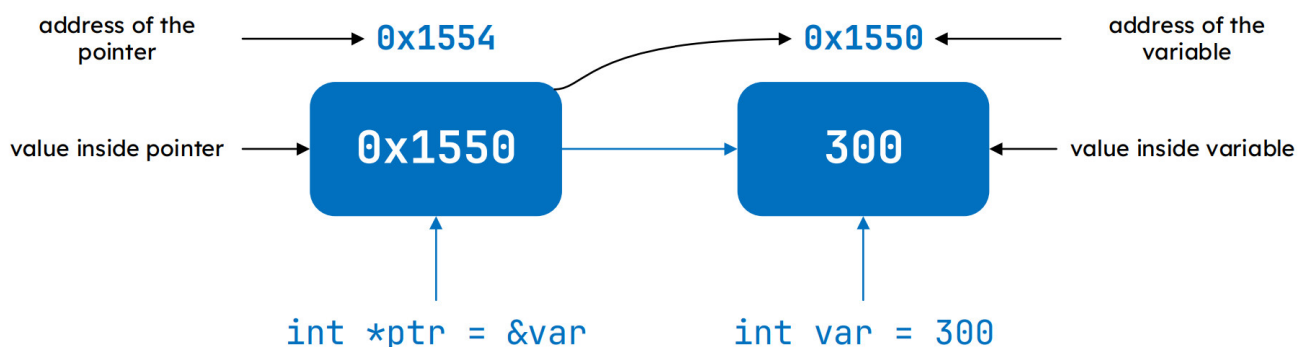
Following is an example of declaration of a Pointer variable:

```
int *ptr;
```

Pointer variable holds the memory address of the variable which is of same data type (integer in this case). To assign the memory address of any variable to the pointer variable we use Address of Operator (&):

```
int var = 300;  
int *ptr = var;
```

In this statement **ptr** now holds the memory address of an integer variable **var**. To access the value at the memory address (currently stored) in the variable we use Dereferencing Operator *. Do not confuse this with the symbol used for the declaration of a pointer.



```
int var1 = *ptr;
```

In this statement another integer variable **var1** is now initialized with the value at the memory address which is stored in **ptr** (that is the value of **var**).

```

#include <iostream>
using namespace std;

int main() {

    int num = 42;

    int* ptr = &num;

    cout << "Value of num: " << num << endl;
    cout << "Address of num: " << &num << endl;
    cout << "Value of num using pointer: " << *ptr << endl;
    cout << "Address stored in pointer: " << ptr << endl;

    *ptr = 100;
    cout << "New value of num: " << num << endl;

    // Pointer arithmetic
    int arr[] = {10, 20, 30, 40, 50};
    int* arrPtr = arr; // point to the first element of the array

    cout << "Array elements using pointer arithmetic:" << endl;
    for (int i = 0; i < 5; ++i) {
        cout << "Element " << i << ": " << *(arrPtr + i) << endl;
    }

    return 0;
}

```

Sample Run

```

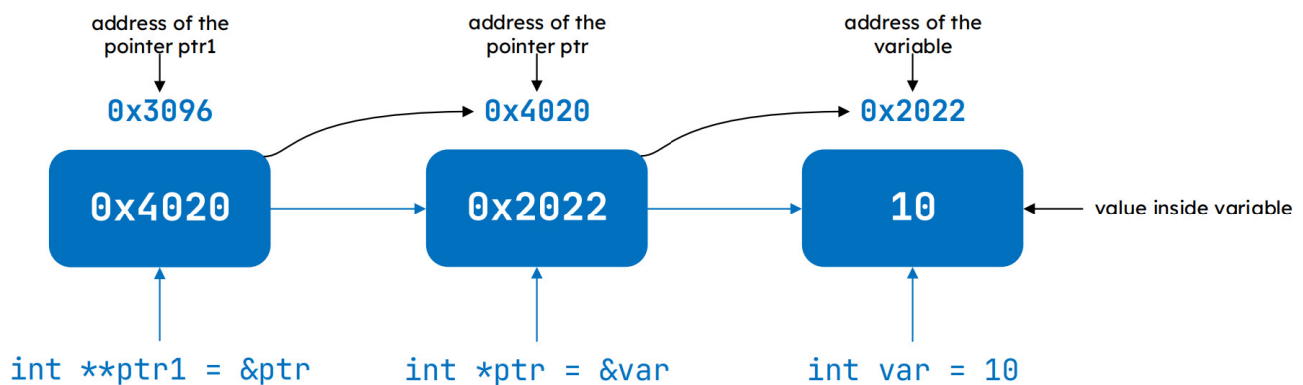
Value of num: 42
Address of num: 0x61ff00
Value of num using pointer: 42
Address stored in pointer: 0x61ff00
New value of num: 100
Array elements using pointer arithmetic:
Element 0: 10
Element 1: 20
Element 2: 30
Element 3: 40
Element 4: 50

```

1.2. 2D-Pointer

In C++, a 2D array can be represented using pointers. A 2D pointer is essentially a pointer to an array of pointers, where each pointer points to a separate array representing a row in the 2D array. Following is an example of declaration of a 2D pointer:

```
int **ptr1;
```

The above diagram shows the memory representation of a pointer to a pointer. The first pointer **ptr** stores the address of the variable **var** and the second pointer **ptr1** stores the address of the first pointer **ptr**.

1.3. Dynamic Memory Allocation

Variables created during the program execution are called dynamic variables. To create a dynamic variable, we use new operator. C++ supports three types of memory allocation:

1. Static memory allocation happens for static and global variables. Memory for these types of variables is allocated once when your program is run and persists throughout the life of your program.
2. Automatic memory allocation happens for function parameters and local variables. Memory for these types of variables is allocated when the relevant block is entered, and freed when the block is exited, as many times as necessary.
3. Dynamic memory allocation is a way for running programs to request memory from the operating system when needed.

1.3.1. new Keyword

This keyword is used to allocate a memory of a particular type. This creates an object using the memory and returns a pointer containing the memory address. The return value is mostly stored in a pointer variable.

```

#include <iostream>
using namespace std;

int main(){

    int *ptr = new int; //allocate memory
    *ptr = 7; // assign value

    int *ptr1 = new int(5); // allocate memory and assign value

}

```

1.3.2. delete Keyword

When we allocate memory dynamically, we need to explicitly tell C++ to deallocate this memory. delete Operator is used to release / deallocate the memory. Consider the below example:

```
#include <iostream>
using namespace std;

int main(){

    int *ptr = new int; //allocate memory
    *ptr = 7; // assign value

    int *ptr1 = new int(5); // allocate memory and assign value

    cout << "Value of ptr: " << *ptr << endl;
    cout << "Address of ptr: " << &ptr << endl;

    cout << "Value of ptr1: " << *ptr1 << endl;
    cout << "Address of ptr1: " << &ptr1 << endl;

}
```

Sample Run	Value of ptr: 7 Address of ptr: 0xfd15b0 Value of ptr1: 5 Address of ptr1: 0xfd6e10
-------------------	--

To delete the dynamically allocated memory we use **delete** keyword. **delete** keyword is used to free the memory which is dynamically allocated using **new** keyword.

```
delete ptrVar; //to deallocate single dynamic variable
delete [] ptrArray; //to deallocate dynamically created array.
```

```
#include <iostream>
using namespace std;

int main(){

    int *ptr = new int; //allocate memory
    *ptr = 7; // assign value

    int *ptr1 = new int(5); // allocate memory and assign value

    cout << "Value of ptr: " << *ptr << endl;
    cout << "Address of ptr: " << ptr << endl;

    cout << "Value of ptr1: " << *ptr1 << endl;
    cout << "Address of ptr: " << ptr1 << endl;

    delete ptr, ptr1;
```

```
}
```

1.3.3. Example 01: Code for 2D-Pointer using Dynamic Memory Allocation

```
#include <iostream>
using namespace std;

int main() {
    int rows, cols;

    cout << "Enter the number of rows: ";
    cin >> rows;
    cout << "Enter the number of columns: ";
    cin >> cols;

    int** ptr = new int*[rows]; // allocate memory for rows (array of pointers)
    for (int i = 0; i < rows; ++i) {
        ptr[i] = new int[cols]; // allocate memory for columns in each row
    }

    // initialize the 2D array with values
    cout << "Enter the elements of the 2D array:" << endl;
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            cout << "Enter element [" << i << "][" << j << "]: ";
            cin >> ptr[i][j];
        }
    }

    cout << "The 2D array is:" << endl;
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            cout << ptr[i][j] << " ";
        }
        cout << endl;
    }

    ptr[1][1] = 99; // change the element at row 1, column 1 to 99
    cout << "After modification, the 2D array is:" << endl;
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            cout << ptr[i][j] << " ";
        }
        cout << endl;
    }

    // deallocate memory to avoid memory leaks
    for (int i = 0; i < rows; ++i) {
        delete[] ptr[i]; // delete each row
    }
}
```

```
delete[] ptr; // delete the array of pointers
```

```
return 0;
```

```
}
```

Sample Run

```
Enter the number of rows: 3
Enter the number of columns: 3
Enter the elements of the 2D array:
Enter element [0][0]: 1
Enter element [0][1]: 2
Enter element [0][2]: 3
Enter element [1][0]: 4
Enter element [1][1]: 5
Enter element [1][2]: 6
Enter element [2][0]: 7
Enter element [2][1]: 8
Enter element [2][2]: 9
The 2D array is:
1 2 3
4 5 6
7 8 9
After modification, the 2D array is:
1 2 3
4 99 6
7 8 9
```

1.3.4. Example 02: Code for 2D-Pointer using Dynamic Memory Allocation with Pointer Arithmetic

```
#include <iostream>
using namespace std;
```

```
int main() {
    int rows, cols;

    cout << "Enter the number of rows: ";
    cin >> rows;
    cout << "Enter the number of columns: ";
    cin >> cols;

    int** ptr = new int*[rows]; // allocate memory for rows (array of pointers)
    for (int i = 0; i < rows; ++i) {
        ptr[i] = new int[cols]; // allocate memory for columns in each row
    }

    // initialize the 2D array with values using pointer arithmetic
    cout << "Enter the elements of the 2D array:" << endl;
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            cout << "Enter element [" << i << "][" << j << "]: ";
            cin >> *(*(ptr + i) + j);
        }
    }
}
```

```

cout << "The 2D array is:" << endl;
for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < cols; ++j) {
        cout << *(*ptr + i) + j << " ";
    }
    cout << endl;
}

*(*ptr + 1) + 1 = 99;
cout << "After modification, the 2D array is:" << endl;
for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < cols; ++j) {
        cout << *(*ptr + i) + j << " ";
    }
    cout << endl;
}

// deallocate memory to avoid memory leaks
for (int i = 0; i < rows; ++i) {
    delete[] ptr[i]; // delete each row
}
delete[] ptr; // delete the array of pointers

return 0;
}

```

Sample Run

```

Enter the number of rows: 3
Enter the number of columns: 3
Enter the elements of the 2D array:
Enter element [0][0]: 1
Enter element [0][1]: 2
Enter element [0][2]: 3
Enter element [1][0]: 4
Enter element [1][1]: 5
Enter element [1][2]: 6
Enter element [2][0]: 7
Enter element [2][1]: 8
Enter element [2][2]: 9
The 2D array is:
1 2 3
4 5 6
7 8 9
After modification, the 2D array is:
1 2 3
4 99 6
7 8 9

```


2. structs in C++

Structure is a collection of variables of different data types under a single name. It is similar to a class in that, both holds a collection of data of different data types. For example: You want to store some information about a **person**: his/her **name**, **citizenship number** and **salary**. You can easily create different variables **name**, **citNo**, **salary** to store this information separately.

However, in the future, you would want to store information about multiple people. Now, you'd need to create different variables for each information per person: **name1**, **citNo1**, **salary1**, **name2**, **citNo2**, **salary2**.

You can easily visualize how big and messy the code would look. Also, since no relation between the variables (information) would exist, it's going to be a daunting task. A better approach will be to have a collection of all related information under a single name **Person**, and use it for every person. Now, the code looks much cleaner, readable and efficient as well. This collection of all related information under a single name **Person** is a structure.

2.1. Declaring a struct in C++

The **struct** keyword defines a structure type followed by an identifier (name of the structure). Then inside the curly braces, you can declare one or more members (declare variables inside curly braces) of that structure. For example:

```
struct Person{  
  
    char department[50];  
    int age;  
    float salary;  
  
};
```

Here a structure **Person** is defined which has three members: **department**, **age** and **salary**.

When a structure is created, no memory is allocated. The structure definition is only the blueprint for the creating of variables. You can imagine it as a datatype like when you define an integer. The **int** specifies that a variable can hold integer element only. Similarly, structure definition only specifies that, what property a structure variable holds when it is defined.

Note: Remember to end the declaration with a semicolon ;.

2.2. Defining a struct Variable in C++

Once you declare a structure **Person** as above. You can define a structure variable as:

```
Person talha;
```

Here, a structure variable **talha** is defined which is of type structure **Person**. When structure variable is defined, only then the required memory is allocated by the compiler. Considering you have either 32-bit or 64-bit system, the memory of float is 4 bytes, memory of **int** is 4 bytes and memory of char is 1 byte. Hence, 58 bytes of memory is allocated for structure variable **talha**.

2.3. Accessing Members of struct in C++

The members of structure variable are accessed using a dot . operator. Suppose, you want to access the **department** member of the structure variable **talha** and assign it the string **AI & DS**. You can perform this task by using following code below:

```
#include <iostream>
#include <cstring>
using namespace std;

struct Person {
    char department[50];
    int age;
    float salary;
};

int main() {
    Person talha;

    strcpy(talha.department, "AI & DS");

    cout << "Department: " << talha.department << endl;

    return 0;
}
```

Sample Run

Department: AI & DS

2.4. Real World Use Case of struct in C++

Consider a real-world scenario of an **Employee** structure in C++ with attributes that might be used in a personal information management system.

```
#include <iostream>
using namespace std;

struct Employee {
    int employeeID;
    string name;
    string department;
};

void displayEmployee(Employee& emp) {
    cout << "Employee ID: " << emp.employeeID << endl;
    cout << "Name: " << emp.name << endl;
    cout << "Department: " << emp.department << endl;
    cout << "-----" << endl;
}

int main() {
```

```

Employee employees[10];
int employeeCount = 0;

// employee 1
employees[employeeCount].employeeID = 101;
employees[employeeCount].name = "Talha Shahid";
employees[employeeCount].department = "AI & DS";
employeeCount++;

// employee 2
employees[employeeCount].employeeID = 102;
employees[employeeCount].name = "Syed Faqrooq Zaidi";
employees[employeeCount].department = "AI & DS (Senior Lecturer)";
employeeCount++;

// display all employees
cout << "Employee Details:" << endl;
for (int i = 0; i < employeeCount; i++) {
    displayEmployee(employees[i]);
}

return 0;
}

```

Sample Run	<pre> Employee Details: Employee ID: 101 Name: Talha Shahid Department: AI & DS ----- Employee ID: 102 Name: Syed Faqrooq Zaidi Department: AI & DS (Senior Lecturer) ----- </pre>
-------------------	--